CENTRE FOR CONTINUING EDUCATION · INDIAN INSTITUTE OF SCIENCE, BENGALURU

Agentic AI Coding

From Specification to Deployment

CCE Short Term Course · Course project

GhumakkadYatri — Project Summary

GhumakkadYatri is a free web app where travellers plan a trip anywhere in the world day by day, with times, and get suggested day plans when they don't know the place. It was built spec-first, then coded, tested and deployed live at ghumakkadyatri.vercel.app.

Name	Priyanka Ghate
Course	Agentic AI Coding: From Specification to Deployment CCE Short Term Course, Centre for Continuing Education, Indian Institute of Science, Bengaluru
Project	This project is about building a smart travel planning web application that helps users design trips across the world. It generates day-wise itineraries based on user-defined timings, allows prioritization by budget or location, and suggests hotel options with map visibility. An integrated AI feature can automatically recommend plans when users are unfamiliar with a destination, making trip planning effortless and personalized.
Professor	**Prof. Deepak Subramani** Associate Professor Computational and Data Sciences Indian Institute of Science
Live app	https://ghumakkadyatri.vercel.app
Code	github.com/PriyankaG5555/PROJECTTEST1

How it was prepared

Every feature started as a written spec, and code was written only after the spec was agreed. Claude Code acted as the AI pair programmer; I made the product decisions.

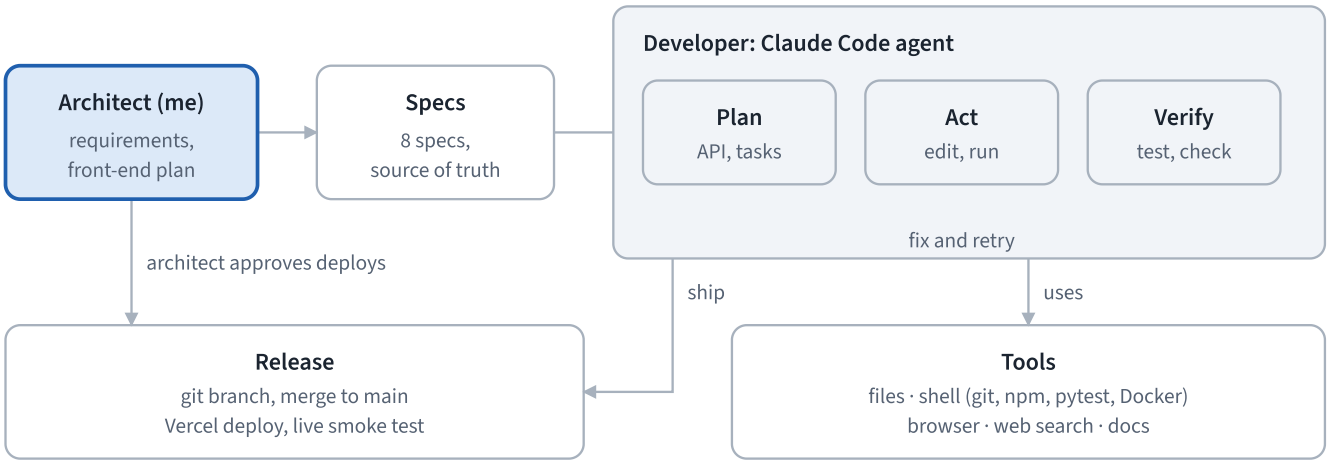
1. **Goal spec** — problem, personas (solo, couple, family, friends), goals, non-goals (no booking, no native apps) and user stories with acceptance criteria.
2. **Layer specs** — front end (screens, states), back end (data model, rules, security) and API contract (every endpoint, request, response and error code).
3. **Implementation plan** — 12 phases (setup → auth → trips → drafts → PDF → first deploy → UI screens → polish), each ending in tests.
4. **Build and verify** — one feature branch per change, tests and lint before merge, then deploy and a live smoke test.
5. **Change through specs** — new needs (AI planning, observability, hotel stays) were added as new specs first, with each decision dated in a change log.

Why this works: the specs are the single source of truth, so the AI writes code against clear rules, and every acceptance criterion maps to a test.

Agentic AI approach

The work starts with the architect: I defined the requirements and planned the front end. The developer, Claude Code (an AI agent), then suggested the API, built the back end and the screens, tested and released, with each step approved by me. One agent played every agent role in turn; no separate sub-agents were used.

Architect sets requirements; the developer agent plans, builds, verifies



STEP	WHO	WHAT THEY DID	CHECKPOINT
1. Architect	Me (Priyanka); the agent drafted specs on request	Requirements: goal, personas, user stories, acceptance criteria. Front-end plan: screens, user flows, states. Key decisions: hosting, no API keys, SigNoz, hotel data	I reviewed every spec and picked between the options offered
2. Developer	Claude Code agent	Suggested the API contract (endpoints, requests, errors) from the front-end plan; built the FastAPI back end, database, planner and PDF; then the React screens. Researched options with web search	I approved the API contract before the back end was built
3. Tester	Claude Code agent	Wrote and ran back-end, front-end and browser tests, lint and type checks; ran the app and read logs; fixed failures	Nothing merged with failing checks
4. Release	Claude Code agent	Branches, merges, production database migrations, deploy check, live smoke test	Only when I said "commit, push, merge and deploy"
Writer	Claude Code agent	READMEs, setup guides and this summary	I reviewed

Guardrails — the agent never put secrets in code or chat, never created accounts or typed passwords, and asked before any action that is hard to undo. When it tried a risky change to SigNoz's login database, a permission check blocked it until I approved.

Timeline

The MVP was planned for 2 weeks with one developer; it was specified, built and deployed in 2 days (2–3 Oct 2026). Three feature rounds followed, finishing on 11 Oct 2026.

DATE (2026)	STAGE	WHAT HAPPENED
2–3 Oct	Requirements (architect)	Spec templates, goal spec, front-end spec, API contract, back-end spec, implementation plan
3 Oct	MVP build and first deploy	Phases 0–11 in one sitting: setup, login, trips, drafts, PDF, screens, browser tests; live on Vercel the same evening
3 Oct	Feature: priority and budget	Trip priority, budget warnings and place suggestions
10 Oct	Feature: AI day planning	Plan my day; the LLM option was replaced by free open data the same evening, about 2 hours from spec to live
11 Oct	Feature: observability and stays	SigNoz with OpenTelemetry traces, metrics and logs; hotel stays and suggestions

Timeline considerations

- **Time-boxed phases** — each of the 12 phases ended in passing tests, so a working app existed at every step and scope could be cut without breaking anything.
- **Specs first** — the first day went mostly on specs; the build then ran fast, with few rewrites.
- **Free tiers, no API keys** — no waiting on approvals or billing; when the LLM needed a key, switching to open data took one evening.
- **Small feature rounds** — each new feature went spec → build → test → deploy within a day, with the live site smoke-tested after each round.

Key design decisions

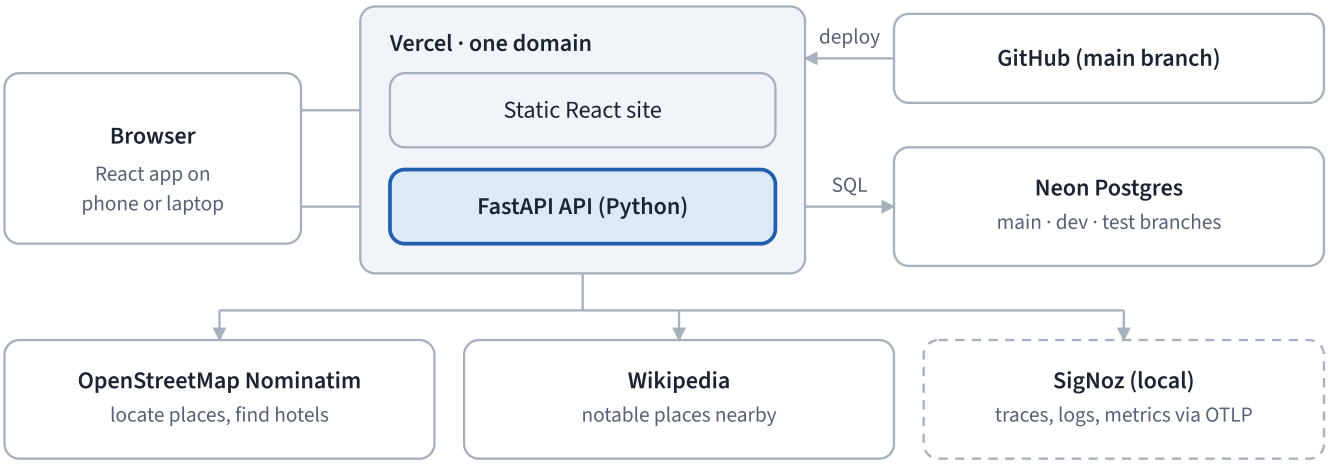
The guiding rule was free, simple and safe: free tiers and open data only, no paid APIs and no API keys.

AREA	CHOICE	WHY
Hosting	Vercel: React site and Python API on one domain	Free, no cold-sleep plan, auto-deploys from GitHub; one domain keeps login cookies simple
Database	Neon Postgres with main, dev and test branches	Free serverless Postgres; tests run on their own branch, never on real data
Back end	Python FastAPI	Typed, fast, generates API docs from code
Login	Username and password, hashed with bcrypt; session in a secure httpOnly cookie	No third-party login needed; the cookie can't be read by page scripts
AI feature	Rule-based planner + Wikipedia + OpenStreetMap, no LLM	Free, no keys to leak, same input gives the same plan, easy to test. Paid Google and LLM options were tried and dropped

AREA	CHOICE	WHY
Planning model	Several drafts per trip; finalize locks a trip, reopen unlocks it	Compare options, then keep one clean final plan
Itinerary	Server-generated PDF (ReportLab)	Same layout on every device
Hotels	Stays per night range; hotel suggestions from OpenStreetMap	Free data; the day plan starts and ends at that night's hotel
Observability	OpenTelemetry + SigNoz, open source, running locally	Traces, logs and metrics with no vendor cost; user IDs hashed for privacy

Architecture

One Vercel domain serves site and API; only free open data is called



The browser talks to one domain; the API stores trips in Neon and calls only free open data (OpenStreetMap, Wikipedia). SigNoz (dashed) is optional and runs locally.

Specification, front end and back end

- **Specification** — 8 specs in the repo: goal, front end, back end, API contract, implementation plan, AI feature, observability and accommodation. Each has acceptance criteria and a dated change log.
- **Front end** — React 19 with TypeScript, Vite, Tailwind and TanStack Query; works on phone and laptop. Screens: sign up / log in, my trips, trip form, day-by-day planner (activities, times, costs, budget bar, stays, Plan my day), compare drafts, and account (with delete account).
- **Back end** — FastAPI REST API (`/api/v1`) with SQLAlchemy and Postgres; 5 database migrations with Alembic. It enforces the rules: each user sees only their own trips, finalized trips are read-only, rate limits on login and suggestions, consistent error codes, and PDF export.

AI feature: Plan my day

For a traveller who doesn't know the place, **Plan my day** builds a timed plan for one day from free open data, and the traveller picks what to keep.

1. **Input** — the day, a time window (e.g. 09:00–18:00), optional interests, must-visit places and places to avoid; plus the trip's priority (Budget, Destinations or Time).
2. **Find places** — OpenStreetMap (Nominatim) locates the destination; Wikipedia lists notable places nearby. Results are cached for 7 days.
3. **Rank** — places are scored by priority and interests, with a bonus for places within 15 km of that night's hotel.
4. **Schedule** — a visit time per type of place, travel time estimated from distance, and the nearest good place next; up to 4 stops, starting from the hotel and leaving time to return.
5. **Review** — the user sees each stop with its reason ("3 km from your hotel"), ticks the ones to keep, and they are saved together; nothing changes until they apply.

Hotel suggestions use the same open data: up to 15 hotels, guest houses and hostels near the destination, ordered by the trip's priority.

Guardrails — no invented prices or opening hours, data sources credited, unverified places labelled, and a daily request limit per user.

Evaluation

Each acceptance criterion in the specs is checked by an automated test, and every release is re-checked on the live site.

CHECK	WHAT IT COVERS	RESULT
Back-end tests (pytest)	API rules, security (no access to others' trips), planner logic, stays, telemetry privacy; run on a separate test database with fake map data	189 passing
Front-end tests (Vitest)	Formatting and helper logic	9 passing
End-to-end test (Playwright)	A full user journey in a real browser	Main journey covered
Code quality	Linting (ruff, oxlint) and strict type checks (mypy, TypeScript)	Clean
Live smoke test	After each deploy: sign up, plan, finalize, export PDF, delete account	Passing
Observability (SigNoz)	Real response times: database queries p95 ≈ 97 ms; outside APIs (Wikipedia, OpenStreetMap) p95 ≈ 1.3 s, which is why results are cached	Measured

Deployment strategy

Pushing to GitHub `main` deploys the live site automatically; everything before that is checked on a branch.

1. **Branch** — each feature is built on its own branch, tested locally against the dev and test databases.
2. **Merge** — tests, lint and type checks pass, then the branch is merged into `main`.

3. **Migrate** — database changes are versioned Alembic migrations, applied to the production database (Neon) with each release.
4. **Deploy** — Vercel builds the React site and the Python API from `main` and serves both on one domain.
5. **Verify** — a smoke test runs the main user journey against the live site.

Secrets stay out of the code: database URLs and keys live in Vercel's settings and a local `.env` file that is never committed.

Observability runs locally: SigNoz in Docker collects traces, logs and metrics from the local app. It is off in production until SigNoz is hosted somewhere the live site can reach (SigNoz Cloud or a server).
